INFORME DE LABORATORIO 09 · TECNOLOGÍA WEB I

# Laboratorio 9 — DNS profesional: vistas (split-horizon), DNSSEC con dnssec-policy y monitoreo

Informe de resultados: configuración, pruebas y evidencias de las dos caras de ns1, la firma del dominio público y su monitoreo

| | |
|---|---|
| **Asignatura** | Tecnología Web I (SIS-0150) · Sexto semestre · Presencial |
| **Master Class** | 09 de 15 — Vistas, DNSSEC y monitoreo |
| **Modalidad** | Individual |
| **Criterio de verificación** | C3 — Explica los componentes y la configuración de un servidor DNS (40 %). |
| **Docente** | M.Sc. Ing. Jimmy Nataniel Requena Llorentty |
| **Estudiante** | Lizeth Rodríguez · Fecha: 22 / 10 / 2026 |
| **Resultado** | `verificar_lab09.sh` → **16 de 16 comprobaciones** ✅ |

## 1. Objetivo

Configurar vistas en ns1 para que el colegio e internet reciban respuestas distintas, firmar la zona pública con `dnssec-policy`, validar la cadena de confianza con `delv` (incluida una falsificación simulada), generar el DS y monitorear el servicio, hasta pasar el verificador de 16 comprobaciones. Este informe documenta cómo se hizo, qué se observó en cada prueba y por qué.

## 2. Contexto del problema

El colegio publicará su dominio real `ue-horizonte.example`. Un consultor de seguridad advirtió dos cosas: "desde internet se ven sus nombres internos" y "su dominio podría ser suplantado con envenenamiento de DNS". La solución fue darle a ns1 dos caras: una **vista interna** para la red del colegio y una **vista externa**, con el dominio público **firmado con DNSSEC**, para el resto del mundo.

**Arquitectura implementada**

```
cliente 10.10.10.50 ─┐                ┌─ vista "interna" (acl red-colegio)
                     ├─▶ ns1 ─┤   horizonte.internal, inversa y
externo 172.30.0.100 ─┘                │   ue-horizonte.example (IPs privadas)
                                       └─ vista "externa" (any)
                                           ue-horizonte.example FIRMADO (DNSSEC)
ns1 = 10.10.10.2 (colegio) y 172.30.0.2 (cara pública)
ns2 10.10.10.3 ◀── AXFR con TSIG (Lab 7): solo las zonas internas
```

## 3. Requisitos previos

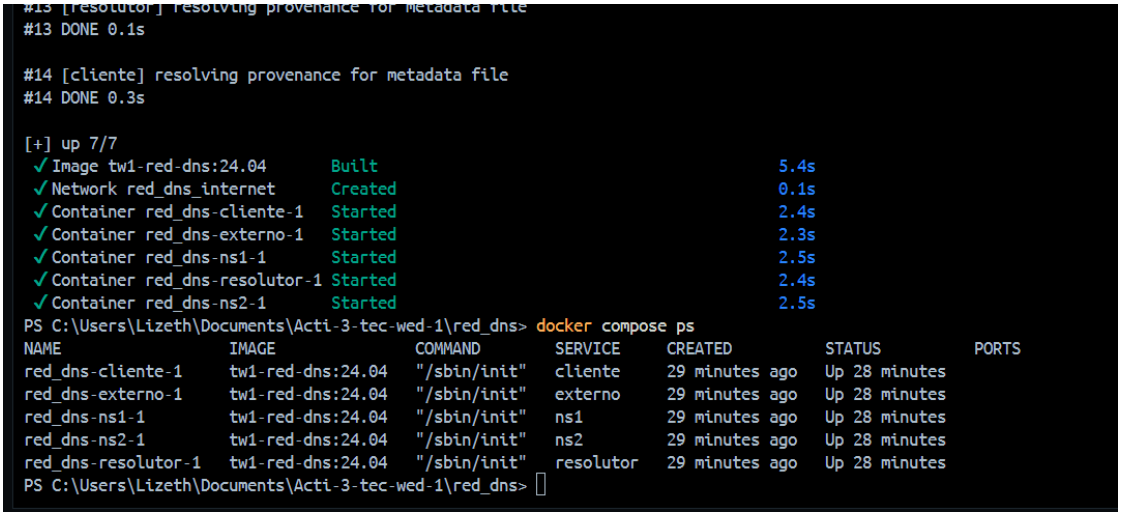
La red `red_dns` no tenía la red `internet` ni el nodo `externo`, así que se agregaron al `compose.yaml` de la carpeta nueva `Acti-3-tec-wed-1`: la red 172.30.0.0/24, ns1 conectado a las dos redes (10.10.10.2 y 172.30.0.2) y `externo` en 172.30.0.100. Con `docker compose up -d --build` quedaron los cinco nodos activos (Figura 1).

`red_dns/compose.yaml (cambios)`

```
networks:
  aula:     { ipv4_address: 10.10.10.2 }
  internet: { ipv4_address: 172.30.0.2 }    # ns1: la "cara pública"
...
externo:
  networks: { internet: { ipv4_address: 172.30.0.100 } }
```

**Figura 1**

*La red red_dns con la red internet y los cinco nodos, incluido externo*




Como ns1 y ns2 se recrearon, se volvió a instalar el Laboratorio 7 (`instalar_ns1.sh` e `instalar_ns2.sh`) y `verificar_lab07.sh` dio **16 de 16** (Figura 2). Antes se normalizaron los fines de línea de Windows con `sed -i 's/\r$//'`.

**Figura 2**

*Laboratorio 7 reinstalado: 16 de 16 comprobaciones*



# 4. Archivos del laboratorio

Se usó la carpeta `lab09` entregada por el docente, montada en todos los nodos como `/lab/lab09`. La Tabla 1 resume qué se completó.

**Tabla 1**

*Archivos del Laboratorio y TODO Completados*

| Archivo | TODO | Qué se completó |
|---|---|---|
| `ns1/named.conf` | 1 | Se borró `include "/etc/bind/named.conf.default-zones";` (zonas fuera de las vistas). |
| `ns1/named.conf.options` | 1 | `nsec3param iterations 0 optout no salt-length 0;` en la política `horizonte` (RFC 9276). |
| `ns1/named.conf.local` | 2 | Vista interna: `match-clients { "red-colegio"; };` · zona externa: `dnssec-policy "horizonte";` e `inline-signing yes;`. |
| `vigilar_dns.sh` | 1 | Cálculo de `dias` hasta el vencimiento del RRSIG con `date -u`. |
| `ns1/db.horizonte.internal` | — | Serial subido a `2026102201` para que ns2 vuelva a copiar la zona (Paso 5). |
| `verificar_lab09.sh` | — | 16 comprobaciones; resultado final **16 de 16**. |

# 5. Desarrollo paso a paso

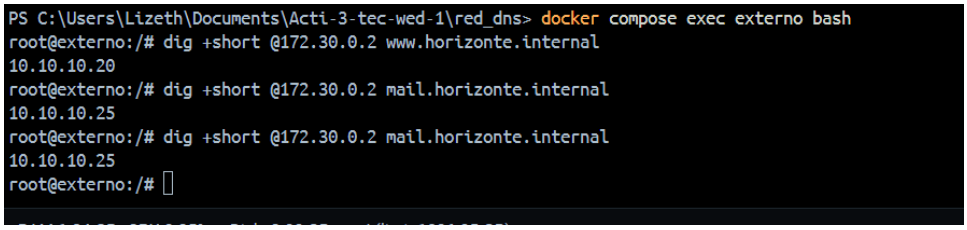
## Paso 1 — La fuga

### Parte A — desde el nodo externo

Antes de configurar las vistas se consultó a ns1 desde `externo`, que simula a cualquier equipo de internet (Figura 3). `www.horizonte.internal` respondió **10.10.10.20** y `mail.horizonte.internal` **10.10.10.25**: alguien de fuera ve el direccionamiento interno y qué servicios tiene el colegio. Eso es reconocimiento gratis para preparar un ataque o un phishing creíble.

**Figura 3**

*Desde internet se ven las IPs internas del colegio*




La transferencia completa (AXFR) dio `Transfer failed` (Figura 4), porque desde el Laboratorio 7 exige la clave TSIG. Aun así, las consultas nombre por nombre sí responden: el problema no es la transferencia, es que ns1 responde lo mismo a todos.

**Figura 4**

*La AXFR sin clave se rechaza (TSIG del Laboratorio 7)*




> 🔎 **Verificado**
>
> Desde externo, `www.horizonte.internal` responde `10.10.10.20` (antes de las vistas).

## Paso 2 — Vistas

### Parte B — named.conf.local

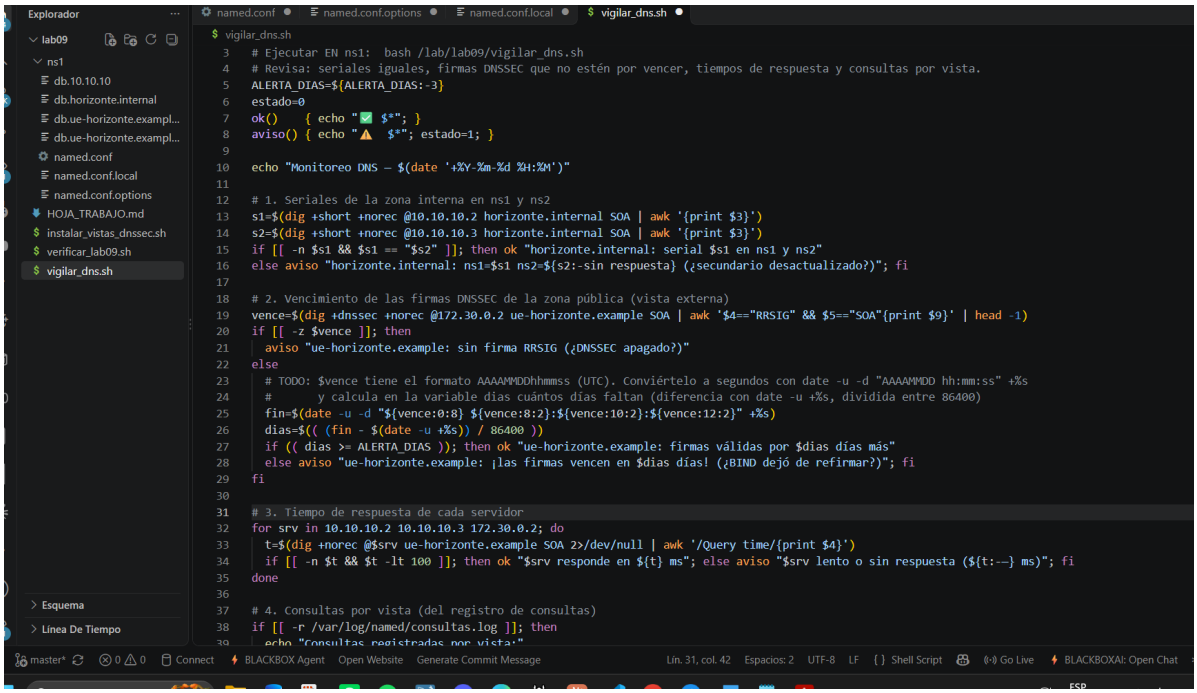
Se completaron los TODO en VS Code (Figura 5). La vista **interna** atiende solo a la `acl "red-colegio"` (10.10.10.0/24 y 127.0.0.0/8) y tiene `horizonte.internal`, su inversa y el dominio público con IPs privadas. La vista **externa** (`match-clients { any; }`) solo tiene el dominio público, firmado. BIND recorre las vistas **en orden** y usa la primera que coincide con la IP de origen, por eso `any` va al final: al revés, atraparía también al colegio.

`ns1/named.conf.local`

```
acl "red-colegio" { 10.10.10.0/24; 127.0.0.0/8; };
view "interna" {
    match-clients { "red-colegio"; };
    zone "horizonte.internal" { type primary; ... allow-transfer { key "transferencia-
horizonte"; }; };
    zone "ue-horizonte.example" { type primary; file "/var/lib/bind/interna/db.ue-
horizonte.example"; };
};
view "externa" {
    match-clients { any; };
    zone "ue-horizonte.example" {
        dnssec-policy "horizonte";
        inline-signing yes;
        type primary;
        file "/var/lib/bind/externa/db.ue-horizonte.example";
    };
};
```

**Figura 5**

*TODO completados en VS Code (las pestañas aún muestran ● sin guardar)*




*Nota*. Los puntos ● de las pestañas indican cambios sin guardar. Por eso las dos primeras instalaciones no tomaron `named.conf` ni `named.conf.options`; se detectó con `cat` y `grep` desde ns1.

La primera instalación se detuvo en la validación con el error que pide la guía. En cuanto existe una `view`, BIND exige que **todas** las zonas estén dentro de alguna vista, y `named.conf.default-zones` de Ubuntu declara fuera las zonas por defecto (`localhost`, `127.in-addr.arpa`…). Como el instalador valida con `named-checkconf -z` antes de reiniciar, BIND nunca quedó caído.

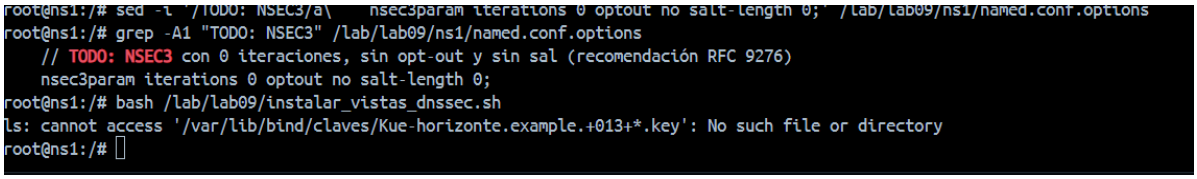
**Error con named.conf tal como viene (salida copiada de la terminal)**

```
root@ns1:/# bash /lab/lab09/instalar_vistas_dnssec.sh
/etc/bind/named.conf.default-zones:2: when using 'view' statements, all zones must be in views
```

`cat` mostró que la línea seguía en el archivo y `grep nsec3param` no encontró nada: VS Code no había guardado. Se borró el `include` con `sed`, se agregó `nsec3param` debajo del TODO y se comprobó con `grep` (Figura 6). Al volver a VS Code apareció un conflicto de guardado (el archivo había cambiado en el disco); se aceptó la versión sin el `include`.

**Figura 6**

*nsec3param agregado y primer intento de instalación*




*Nota*. En este intento BIND no terminó de generar la clave dentro de los 20 s que espera el script (`ls: cannot access … .key`). Al ejecutarlo otra vez terminó correctamente y mostró el DS (Paso 3).

Con la configuración aplicada, la misma consulta da respuestas distintas según quién pregunta (Figuras 7 y 8 y Tabla 2).

**Figura 7**

*Desde el cliente (vista interna): IPs privadas e intranet*

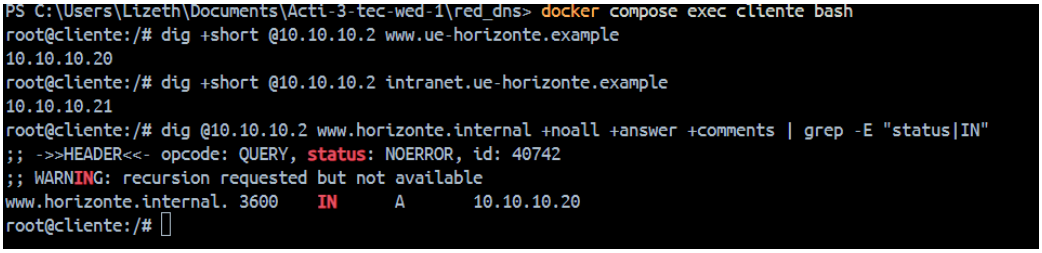



**Figura 8**

*Desde externo (vista externa): IP pública, NXDOMAIN, REFUSED y la firma RRSIG*

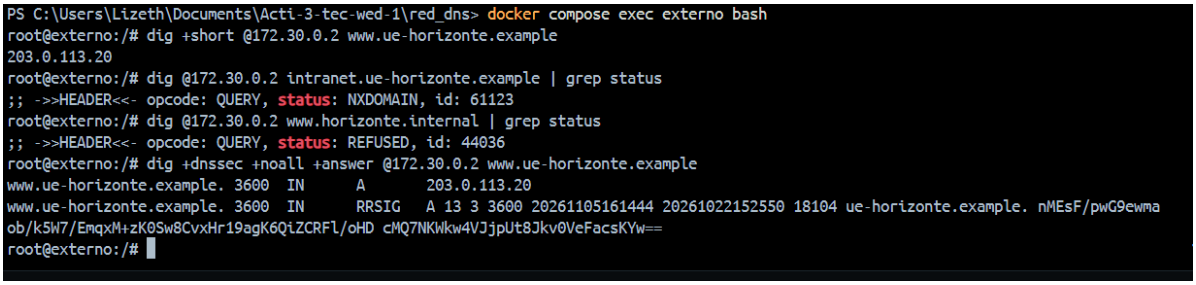



**Tabla 2**

*Parte B: El Mismo Nombre, Dos Respuestas*

| Consulta | Desde el cliente (10.10.10.50) | Desde externo (172.30.0.100) |
|---|---|---|
| `www.ue-horizonte.example` | `10.10.10.20` | `203.0.113.20` |
| `intranet.ue-horizonte.example` | `10.10.10.21` | `NXDOMAIN` |
| `www.horizonte.internal` | `10.10.10.20` (`NOERROR`) | `REFUSED` |

*Nota.* El colegio llega a su web por la IP privada sin salir a internet. Para internet la intranet no existe y `horizonte.internal` no se revela: la vista externa no tiene esa zona.

> 🔎 **Verificado**
>
> Colegio: `10.10.10.20` · Internet: `203.0.113.20` y `REFUSED` para `horizonte.internal`.

## Paso 3 — Firmar con dnssec-policy

### Parte C — política y firma

La política `horizonte` usa una sola clave **CSK** ECDSA P-256 (algoritmo 13), firmas de 14 días que se renuevan cada 5 y **NSEC3 con 0 iteraciones, sin opt-out y sin sal**, como recomienda el RFC 9276: las iteraciones y la sal no dan seguridad real y solo cuestan CPU a los servidores y a los validadores.

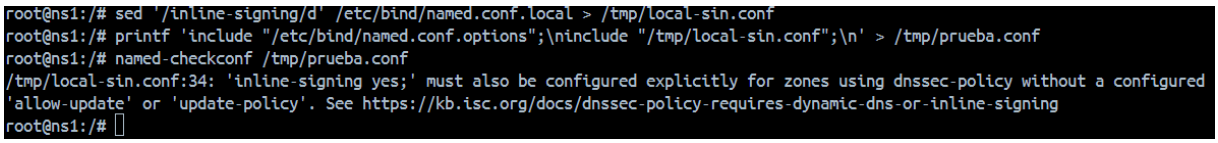
`ns1/named.conf.options`

```
dnssec-policy "horizonte" {
    keys { csk key-directory lifetime unlimited algorithm ecdsap256sha256; };
    nsec3param iterations 0 optout no salt-length 0;
    signatures-validity 14d;
    signatures-refresh 5d;
    max-zone-ttl 1d;
};
```

Para ver qué pasa sin `inline-signing` se probó sobre una **copia** en `/tmp`, sin tocar la configuración real (Figura 9). En BIND 9.18, `dnssec-policy` necesita escribir en la zona; como nuestra zona es un archivo estático sin actualizaciones dinámicas, hay que pedir `inline-signing`. Con eso BIND deja intacto el archivo escrito a mano y mantiene aparte una copia firmada que crea, firma y re-firma solo.

**Figura 9**

*Sin inline-signing, named-checkconf rechaza la zona firmada*




BIND generó la clave `18104` en `/var/lib/bind/claves` y la zona `.signed` con sus diarios en `/var/lib/bind/externa` (Figura 10). El `.key` es la clave pública (DNSKEY), el `.state` guarda el estado para la política y el `.private` es la clave privada con la que firma. Del `.private` solo se muestra el nombre: no se abre, no se copia y no se sube al repositorio.

**Figura 10**

*Clave y zona firmada generadas por BIND*




Desde externo, `dig +dnssec` devuelve el registro A acompañado de su **RRSIG** (Figura 8). La Tabla 3 explica cada campo.

**Tabla 3**

*Campos del RRSIG de www.ue-horizonte.example*

| Campo | Valor | Significado |
|---|---|---|
| **Tipo cubierto** | `A` | Firma el registro A de `www`. |
| **Algoritmo** | `13` | ECDSA P-256 con SHA-256 (la CSK de la política). |
| **Etiquetas** | `3` | `www`, `ue-horizonte`, `example`. |
| **TTL original** | `3600` | El TTL con el que se firmó el registro. |
| **Vencimiento** | `20261105161444` | 05/11/2026 16:14:44 UTC, unos 14 días (`signatures-validity 14d`). |
| **Inicio** | `20261022152550` | 22/10/2026 15:25:50 UTC: una hora antes de firmar (16:25:50), para tolerar relojes desfasados. |

| Campo | Valor | Significado |
|---|---|---|
| **Etiqueta de clave** | `18104` | La misma del `.key`, del DS y de `rndc dnssec -status`. |
| **Firmante** | `ue-horizonte.example.` | La zona que firma. |

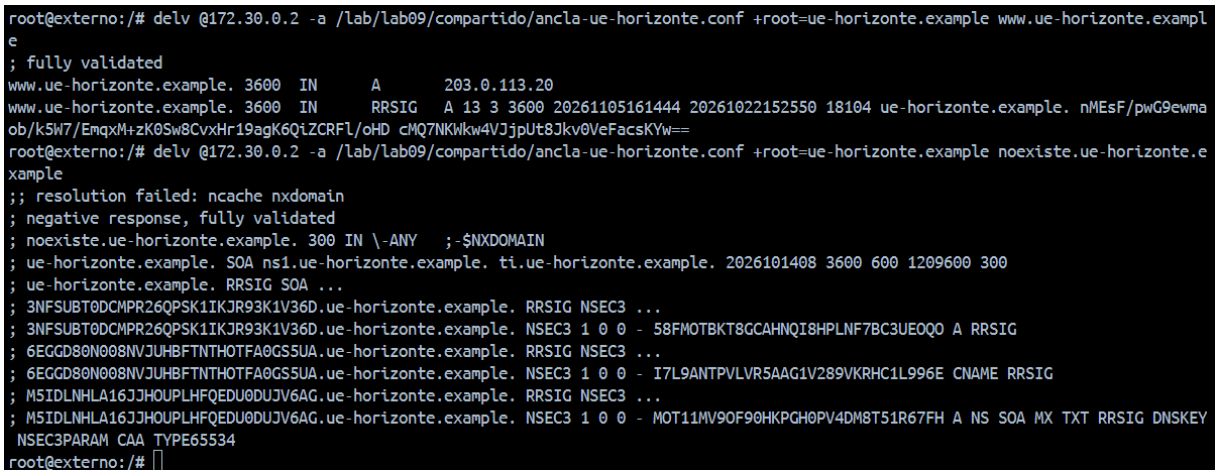
> 🔎 **Verificado**
> `dig +dnssec @172.30.0.2 www.ue-horizonte.example` muestra `RRSIG A 13 3 3600 ….`

## Paso 4 — Validar y romper

### Parte C — delv

`delv` valida como lo haría un resolutor, usando como **ancla de confianza** la clave pública que generó el instalador (Figura 11). Para `www` respondió **`; fully validated`**: el ancla valida el DNSKEY de la zona y ese DNSKEY valida el RRSIG del registro, así que la respuesta es auténtica. Para `noexiste` respondió **`; negative response, fully validated`**: lo que no existe se prueba con **NSEC3**, registros firmados que dicen "entre este hash y el siguiente no hay ningún nombre". La salida muestra `NSEC3 1 0 0 -`: SHA-1, sin opt-out, 0 iteraciones y sin sal, tal como se configuró.

**Figura 11**
*delv: respuesta y NXDOMAIN validados (NSEC3)*




Después se cambió **una sola letra** de la clave en una copia del ancla (`jN/9…` → `AN/9…`) y se repitió (Figura 12). delv respondió `no valid signature found (DS)` y **`broken trust chain`**, y no entregó ninguna IP. Esto demuestra lo que pedía el consultor: si alguien envenena una caché con respuestas falsas, no puede firmarlas sin la clave privada, y un resolutor que valida las rechaza en lugar de engañar al usuario.

**Figura 12**

*Ancla con una letra cambiada: broken trust chain*



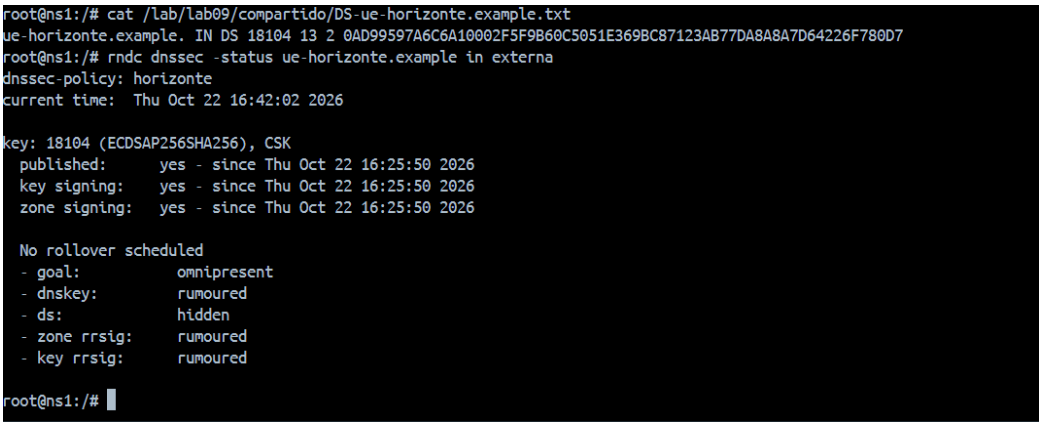

El DS es el resumen SHA-256 (tipo `2`) de la clave `18104`. Se entrega al **registrador** para que lo publique en la zona padre; así la cadena empieza en la raíz y nadie necesita configurar un ancla a mano. `rndc dnssec -status` confirma la política, la CSK firmando desde las 16:25:50 UTC y **`ds: hidden`**: BIND sabe que el DS todavía no está en el padre (Figura 13).

`compartido/DS-ue-horizonte.example.txt`

```
ue-horizonte.example. IN DS 18104 13 2
0AD99597A6C6A10002F5F9B60C5051E369BC87123AB77DA8A8A7D64226F780D7
```

**Figura 13**

*DS para el registrador y estado de la clave con rndc dnssec -status*




> 🔎 **Verificado**
>
> Tres salidas de `delv` (válida, NXDOMAIN validado y ancla falsa) y el DS.

## Paso 5 — Monitoreo

### Parte D — vigilar_dns.sh

El TODO convierte el vencimiento del RRSIG (`AAAAMMDDhhmmss`, en UTC) a segundos y calcula los días que faltan:

`vigilar_dns.sh (TODO)`

```
fin=$(date -u -d "${vence:0:8} ${vence:8:2}:${vence:10:2}:${vence:12:2}" +%s)
dias=$(( (fin - $(date -u +%s)) / 86400 ))
```

El verificador dio primero **15 de 16**: solo fallaba el monitoreo (Figura 14). `vigilar_dns.sh` avisó `ns1=2026100701 ns2=2026100703`. La zona interna del Lab 9 traía un serial **menor** que el que ns2 conservaba del Lab 7, y un secundario solo copia la zona si el serial del primario es **mayor**. Se subió el serial a `2026102201` (la fecha de hoy) y, como ahora la zona vive dentro de una vista, se recargó con `rndc reload horizonte.internal in interna`. Los dos servidores mostraron el mismo serial y el monitoreo quedó sin alertas (Figura 15).

**Figura 14**

*Primer resultado: 15 de 16 (alerta del monitoreo)*




**Figura 15**

*Serial corregido, ns2 sincronizado y monitoreo sin alertas*




El monitoreo informa las firmas válidas por **13 días más**, los tres servidores respondiendo en 4 ms, **30** consultas de la vista externa y **58** de la interna, y el DS todavía `hidden`. Si BIND se detuviera 3 semanas, nadie renovaría las firmas: a los 14 días vencerían y todo resolutor que valide respondería **SERVFAIL**. El dominio "desaparecería" aunque los datos sigan ahí, por eso la alerta salta cuando quedan menos de 3 días. Con todo corregido, el verificador aprobó las **16 comprobaciones** (Figura 16).

**Figura 16**

*verificar_lab09.sh: 16 de 16 comprobaciones*




> 🔎 **Verificado**
> ✅ Monitoreo sin alertas · **Resultado: 16 de 16 comprobaciones.**

## 6. Retos (para llegar a Estratégico)

Los retos opcionales **no se realizaron** en esta entrega. Se dejan indicados con lo que haría falta para cada uno.

**Tabla 4**

*Estado de los Retos*

| Reto | Estado | Qué haría falta |
|---|---|---|
| **`vigilar_dns.sh` con un servicio y un timer de systemd cada hora** | No realizado | Un `vigilar-dns.service` (`Type=oneshot`) y un `vigilar-dns.timer` con `OnCalendar=hourly`; las alertas quedarían en `journalctl -u vigilar-dns`. |
| **ns2 sirviendo también la vista externa** | No realizado | Dos claves TSIG y `match-clients { key "…"; }` en cada vista, para que ns2 transfiera cada vista con su propia clave. |
| **Simular el vencimiento (`signatures-validity 1h`)** | No realizado | Bajar la validez en una copia de la política y observar en `journalctl -u named` cómo BIND re-firma. |

## 7. Uso de Inteligencia Artificial en este laboratorio

> ✅ **Permitido — cómo se usó**
> - Explicaciones de vistas, RRSIG, DS y NSEC3, contrastadas con las salidas reales de `dig` y `delv`.
> - Interpretación de los errores de `named-checkconf` (default-zones e inline-signing) y del aviso de seriales.
> - Cálculo de fechas en bash, probado con `vigilar_dns.sh` (13 días hasta el 05/11/2026).

> ⛔ **No permitido — cómo se respetó**
> - El `.private` de la clave **no** se compartió: solo se listó su nombre con `ls`.
> - La firma la hace `dnssec-policy`; no se usó `dnssec-signzone` + cron.
> - La validación **nunca** se desactivó; la prueba del ancla falsa se hizo con una copia en `/tmp`.

La bitácora completa, con las 12 consultas realizadas, cómo se verificó cada una y qué se aprendió, está en el **Anexo B**.

## 8. Entregable

**Tabla 5**

*Lista de Comprobación del Entregable*

| Elemento | Estado | Evidencia |
|---|---|---|
| **`ns1/` y `vigilar_dns.sh` completos** | ✅ | 5 de 5 TODO (Tabla 1). |
| **Captura del verificador con 16 de 16** | ✅ | Figura 16 |
| **Capturas de `delv` (válida y ancla falsa)** | ✅ | Figuras 11 y 12 |
| **`HOJA_TRABAJO.md` (partes A a D)** | ✅ | Anexo A |
| **`BITACORA_IA.md`** | ✅ | Anexo B |

## 9. Rúbrica de evaluación: evidencias por criterio

**Tabla 6**

*Criterios de la Rúbrica y Dónde se Evidencian*

| Criterio | Puntos | Evidencia en este informe |
|---|---|---|
| **Vistas** | 25 | Respuestas distintas por público (Tabla 2); orden de las vistas y acl explicados; error de `default-zones` explicado (Paso 2). |
| **DNSSEC con dnssec-policy** | 25 | Zona firmada (Figuras 8 y 10); `inline-signing` explicado (Figura 9); RRSIG interpretado (Tabla 3). |
| **Cadena de confianza** | 20 | `delv` válido, NXDOMAIN con NSEC3 y ancla falsa (Figuras 11 y 12); DS y su destino (Figura 13). |
| **Monitoreo** | 15 | Cálculo de vencimiento correcto; alerta de seriales diagnosticada y corregida; sin alertas (Figura 15). |
| **Bitácora y reflexión** | 15 | DNSSEC vs HTTPS (Conclusiones y Anexo A); bitácora verificada (Anexo B). |

## 10. Conclusiones

Las dos advertencias del consultor quedaron resueltas. Desde internet ya no se ven los nombres internos: la misma pregunta recibe la IP privada si viene del colegio y la pública si viene de fuera, mientras que `horizonte.internal` y la intranet no existen para el mundo. El dominio público está firmado: cualquier

resolutor que valide puede comprobar que la respuesta es auténtica, y con una sola letra cambiada en el ancla la cadena se rompe. `dnssec-policy` genera, firma y re-firma sin intervención, y `vigilar_dns.sh` avisa antes de que las firmas venzan o de que ns2 se desincronice.

DNSSEC y HTTPS se complementan. HTTPS protege la conexión una vez que se sabe a qué IP ir; DNSSEC protege que esa IP sea la verdadera, y también los registros que no son web (MX, SPF, DMARC, CAA) y la inexistencia de un nombre. Muchos dominios aún no lo usan porque exige mantener claves y coordinar el DS con el registrador, y un error deja el dominio inaccesible para todos los que validan. Falta un paso para que la protección llegue a todos los visitantes: entregar el DS al registrador, porque mientras esté `hidden` solo valida quien tiene el ancla manual.

## Referencias

Arends, R., Austein, R., Larson, M., Massey, D., & Rose, S. (2005). *DNS security introduction and requirements* (RFC 4033). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc4033

Hardaker, W., & Dukhovni, V. (2022). *Guidance for NSEC3 parameter settings* (RFC 9276). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc9276

Internet Systems Consortium. (s.f.-a). *DNSSEC guide*. BIND 9 Administrator Reference Manual. https://bind9.readthedocs.io/en/latest/dnssec-guide.html

Internet Systems Consortium. (s.f.-b). *dnssec-policy requires dynamic DNS or inline-signing*. ISC Knowledgebase. https://kb.isc.org/docs/dnssec-policy-requires-dynamic-dns-or-inline-signing

Laurie, B., Sisson, G., Arends, R., & Blacka, D. (2008). *DNS security (DNSSEC) hashed authenticated denial of existence* (RFC 5155). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc5155

Requena Llorentty, J. N. (2026). *Guía de laboratorio 09: DNS profesional: vistas (split-horizon), DNSSEC con dnssec-policy y monitoreo* [Guía de laboratorio]. Universidad Privada Domingo Savio.

# Anexo A — HOJA_TRABAJO.md

*Laboratorio 9 — Hoja de trabajo: vistas (split-horizon), DNSSEC y monitoreo.*

Nombre: Lizeth Rodríguez  Fecha: 22/10/2026

Red `red_dns` actualizada con la red `internet` (172.30.0.0/24) y el nodo `externo` (172.30.0.100). ns1 tiene dos caras: 10.10.10.2 (colegio) y 172.30.0.2 (pública). Al recrearse ns1 se reinstaló el Laboratorio 7 (`instalar_ns1.sh` y `instalar_ns2.sh`) y `verificar_lab07.sh` dio **16 de 16**. Carpeta de trabajo: `/lab/lab09`.

## Parte A — El problema

Desde el nodo **externo** (simula internet), antes de las vistas:

| Consulta | Respuesta |
|---|---|
| `dig +short @172.30.0.2 www.horizonte.internal` | `10.10.10.20` |
| `dig +short @172.30.0.2 mail.horizonte.internal` | `10.10.10.25` |
| `dig @172.30.0.2 horizonte.internal AXFR` | `; Transfer failed.` |

**¿Qué información interna ve alguien de fuera?** Cualquiera en internet puede preguntar nombres del colegio y recibir sus IPs privadas: el servidor web (10.10.10.20), el correo (10.10.10.25) y, adivinando nombres, otros equipos. La transferencia completa (AXFR) **no** salió porque desde el Laboratorio 7 exige la clave TSIG, pero las consultas una por una sí responden.

**¿Por qué es un riesgo?** Es reconocimiento gratis para un atacante: conoce el direccionamiento interno, qué servicios hay y cómo se llaman, sin tocar la red. Esa información sirve para preparar ataques dirigidos o phishing creíble. Además, `horizonte.internal` no tiene ningún sentido fuera del colegio.

## Parte B — Vistas

TODO completados:

| Archivo | Qué se escribió |
|---|---|
| `ns1/named.conf` | Se **borró** `include "/etc/bind/named.conf.default-zones";` |
| `ns1/named.conf.local` (vista interna) | `match-clients { "red-colegio"; };` |
| `ns1/named.conf.local` (zona externa) | `dnssec-policy "horizonte";` y `inline-signing yes;` |
| `ns1/named.conf.options` | `nsec3param iterations 0 optout no salt-length 0;` |

### 1. Error con `named.conf` tal como viene:

```
/etc/bind/named.conf.default-zones:2: when using 'view' statements, all zones must be in views
```

Ocurre porque, en cuanto existe una sola `view`, BIND exige que **todas** las zonas estén dentro de alguna vista. `named.conf.default-zones` de Ubuntu declara fuera de las vistas las zonas por defecto (`localhost`, `127.in-addr.arpa`, `0.in-addr.arpa`, `255.in-addr.arpa`). El instalador valida con `named-checkconf -z` y se detuvo antes de reiniciar BIND, así que el servicio nunca quedó caído. Se borró la línea y se volvió a instalar.

**Orden de las vistas.** BIND recorre las vistas **en orden** y usa la **primera** cuyo `match-clients` coincide con la IP de origen. La interna (`acl "red-colegio" { 10.10.10.0/24; 127.0.0.0/8; }`) va primero y la externa (`match-clients { any; }`) al final. Si estuvieran al revés, `any` atraparía también al colegio y nadie vería la vista interna.

**2. Tabla (el mismo nombre, dos respuestas):**

| Consulta | Desde el cliente (10.10.10.50) | Desde externo (172.30.0.100) |
|---|---|---|
| `www.ue-horizonte.example` | `10.10.10.20` | `203.0.113.20` |
| `intranet.ue-horizonte.example` | `10.10.10.21` | `NXDOMAIN` |
| `www.horizonte.internal` | `10.10.10.20` (`NOERROR`) | `REFUSED` |

El colegio llega a su web por la IP privada sin salir a internet y ve la intranet. Internet solo ve la IP pública, la intranet "no existe" para ellos y `horizonte.internal` no se le revela: la vista externa ni siquiera tiene esa zona, por eso responde `REFUSED`.

## Parte C — DNSSEC con dnssec-policy

**1. Sin `inline-signing yes;`**, `named-checkconf` (probado sobre una copia en `/tmp`) dice:

```
'inline-signing yes;' must also be configured explicitly for zones using dnssec-policy without
a configured
'allow-update' or 'update-policy'.
```

En BIND 9.18, `dnssec-policy` necesita poder escribir en la zona. Como nuestra zona es un archivo estático (sin actualizaciones dinámicas), hay que pedir `inline-signing`: BIND deja **intacto** el archivo que escribimos (`db.ue-horizonte.example`) y mantiene aparte una copia firmada (`db.ue-horizonte.example.signed`) que crea, firma y re-firma solo.

**2. Archivos que generó BIND:**

```
/var/lib/bind/claves:
Kue-horizonte.example.+013+18104.key  Kue-horizonte.example.+013+18104.private  Kue-
horizonte.example.+013+18104.state
/var/lib/bind/externa:
db.ue-horizonte.example  db.ue-horizonte.example.jbk  db.ue-horizonte.example.signed  db.ue-
horizonte.example.signed.jnl
```

- `.key`: la clave **pública** (DNSKEY). `+013` = algoritmo 13 (ECDSA P-256) y `18104` = etiqueta de la clave.
- `.private`: la clave **privada** con la que BIND firma. No se muestra, no se copia y no se sube al repositorio.
- `.state`: el estado de la clave para `dnssec-policy` (publicada, firmando, DS…).
- `.signed`: la zona firmada que BIND sirve; `.jnl` y `.jbk`: diarios de cambios de la firma en línea.

**3. RRSIG** (`dig +dnssec @172.30.0.2 www.ue-horizonte.example`):

```
www.ue-horizonte.example. 3600 IN RRSIG A 13 3 3600 20261105161444 20261022152550 18104 ue-
horizonte.example. nMEsF/...
```

| Campo | Valor | Significado |
|---|---|---|
| Tipo cubierto | `A` | Firma el registro A de `www` |
| Algoritmo | `13` | ECDSA P-256 con SHA-256 (la CSK de la política) |
| Etiquetas | `3` | `www`, `ue-horizonte`, `example` |

| Campo | Valor | Significado |
|---|---|---|
| TTL original | `3600` | El TTL con el que se firmó |
| Vencimiento | `20261105161444` | 05/11/2026 16:14:44 UTC (unos 14 días: `signatures-validity 14d`) |
| Inicio | `20261022152550` | 22/10/2026 15:25:50 UTC; BIND la fecha una hora antes de firmar para tolerar relojes desfasados |
| Etiqueta de clave | `18104` | La misma clave del `.key` y del DS |
| Firmante | `ue-horizonte.example.` | La zona que firma |

**4. delv con el ancla:**

```
delv @172.30.0.2 -a /lab/lab09/compartido/ancla-ue-horizonte.conf +root=ue-horizonte.example
www.ue-horizonte.example
; fully validated
www.ue-horizonte.example. 3600 IN A 203.0.113.20
```

"Fully validated" significa que delv comprobó la cadena completa: el ancla de confianza (nuestra clave pública) valida el DNSKEY de la zona, y ese DNSKEY valida el RRSIG del registro A. La respuesta es auténtica y no fue alterada en el camino.

Con `noexiste.ue-horizonte.example` → `; negative response, fully validated`. **Lo que no existe se firma con NSEC3**: la zona tiene una cadena ordenada de *hashes* de los nombres que sí existen, y cada registro NSEC3 (firmado) dice "entre este hash y el siguiente no hay nada". El servidor entrega los NSEC3 que cubren el hueco donde estaría `noexiste` y así prueba su inexistencia sin firmar en el momento. En la salida se ve `NSEC3 1 0 0 -`: SHA-1, sin opt-out, **0 iteraciones** y **sin sal (`-`)**, tal como se configuró (RFC 9276).

**5. Ancla falsa** (copia en `/tmp` con una sola letra cambiada, `jN/9…` → `AN/9…`):

```
;; validating ue-horizonte.example/DNSKEY: no valid signature found (DS)
;; broken trust chain resolving 'www.ue-horizonte.example/A/IN': 172.30.0.2#53
;; resolution failed: broken trust chain
```

La clave del ancla ya no coincide con la DNSKEY que firma la zona, así que la cadena se rompe y delv **no entrega ninguna respuesta**. Esto demuestra que un atacante que envenene la caché con respuestas falsas no puede firmarlas: sin la clave privada correcta, un resolutor que valida las rechaza (SERVFAIL) en lugar de engañar al usuario.

**6. DS:** `compartido/DS-ue-horizonte.example.txt`

```
ue-horizonte.example. IN DS 18104 13 2
0AD99597A6C6A10002F5F9B60C5051E369BC87123AB77DA8A8A7D64226F780D7
```

Es el resumen SHA-256 (tipo `2`) de la clave `18104`, algoritmo `13`. Se entrega al **registrador / zona padre** (en Bolivia, NIC Bolivia para `.bo`) para que lo publique en la zona superior. Así la cadena de confianza empieza en la raíz firmada y llega hasta nuestro dominio sin que cada usuario tenga que configurar un ancla a mano.

`rndc dnssec -status ue-horizonte.example in externa` → `dnssec-policy: horizonte`, clave `18104 (ECDSAP256SHA256), CSK`, `published: yes`, `key signing: yes`, `zone signing: yes`, `No rollover scheduled` y **`ds: hidden`**: BIND sabe que el DS todavía **no** está publicado en el padre. Mientras siga `hidden`, DNSSEC solo protege a quien use el ancla manual (como delv aquí).

## Parte D — Monitoreo

TODO de `vigilar_dns.sh`:

```
fin=$(date -u -d "${vence:0:8} ${vence:8:2}:${vence:10:2}:${vence:12:2}" +%s)
dias=$(( (fin - $(date -u +%s)) / 86400 ))
```

`${vence:0:8}` toma `AAAAMMDD` y los otros cortes arman `hh:mm:ss`. `date -u` lo pasa a segundos (UTC, igual que el RRSIG), se resta la hora actual y se divide entre 86 400 segundos por día.

Primera ejecución: ⚠️ **`horizonte.internal: ns1=2026100701 ns2=2026100703`**. La zona interna del Lab 9 traía el serial 2026100701, **menor** que el 2026100703 que ns2 tenía del Lab 7. Un secundario solo copia la zona si el primario tiene un serial **mayor**, así que ns2 se quedaba con la versión vieja. Se subió el serial a **2026102201** (fecha de hoy) y se recargó con `rndc reload horizonte.internal in interna`. Los dos servidores mostraron 2026102201.

Segunda ejecución: serial igual en ns1 y ns2 ✅, firmas válidas por **13 días más** ✅, los tres servidores responden en 4 ms ✅, consultas por vista: **30 externa / 58 interna**, y ℹ️ `DS en el padre: hidden`.

`bash /lab/lab09/verificar_lab09.sh` → **Resultado: 16 de 16 comprobaciones.**

**¿Qué pasaría si BIND se detiene 3 semanas (firmas de 14 días)?** Las firmas se renuevan cada 5 días (`signatures-refresh`), pero solo si BIND está funcionando. Con BIND detenido, las firmas que ya circulan (en los secundarios o cachés) vencen como máximo a los 14 días. Desde ese momento, todo resolutor que valide DNSSEC responderá **SERVFAIL**: el dominio "desaparece" para esos usuarios, aunque los datos sigan ahí. Por eso el monitoreo avisa cuando quedan menos de 3 días (`ALERTA_DIAS`).

## Reflexión

**¿Qué protege DNSSEC que HTTPS no protege?** HTTPS protege la conexión **después** de saber a qué IP ir. DNSSEC protege **la respuesta del DNS**: que la IP sea la auténtica y no una inyectada por envenenamiento de caché. También cubre lo que no es web (correo con MX, registros TXT como SPF/DMARC, CAA) y prueba que un nombre no existe. HTTPS detecta un sitio falso por el certificado, pero solo si el usuario no ignora la advertencia, y no protege los demás registros DNS.

**¿Por qué muchos dominios aún no lo usan?** Porque agrega trabajo y riesgo de operación: hay que mantener claves, re-firmar y coordinar el DS con el registrador. Si algo falla (firmas vencidas, DS que no coincide), el dominio deja de resolverse para todos los que validan, y eso es peor que no tener DNSSEC. Además, no todos los registradores lo facilitan y muchos ven HTTPS como "suficiente". `dnssec-policy` reduce mucho ese trabajo porque BIND genera, firma y re-firma solo.

# Anexo B — BITACORA_IA.md

*Bitácora de uso de IA — Laboratorio 9 (vistas, DNSSEC con dnssec-policy y monitoreo).*

Estudiante: Lizeth Rodríguez · Entorno: red Docker `red_dns` (ns1 10.10.10.2 / 172.30.0.2, ns2, cliente, externo 172.30.0.100), Ubuntu 24.04, BIND 9.18 · Fecha: 22/10/2026

Reglas que respeté (sección 7 de la guía):

- Los archivos `.private` de la clave DNSSEC **nunca** se mostraron, copiaron ni compartieron con la IA. Solo se listó su nombre con `ls`. Lo que se compartió es público: el DS y el ancla (clave pública).
- Se usó `dnssec-policy` (firma y re-firma automática), no el método manual `dnssec-signzone` + cron.
- **No** se desactivó la validación en ningún momento. La prueba del ancla falsa se hizo con una copia en `/tmp` y el resultado esperado era justamente que fallara.
- Cada respuesta de la IA la comprobé con `named-checkconf`, `dig`, `delv`, `grep` o el verificador.

| # | Qué pregunté | Qué respondió la IA | Cómo lo verifiqué | Qué aprendí |
|---|---|---|---|---|
| 1 | Cómo agregar a `red_dns` la red `internet` y el nodo `externo` que pide la guía. | Un `compose.yaml` con la red 172.30.0.0/24, ns1 en las dos redes (10.10.10.2 y 172.30.0.2) y `externo` en 172.30.0.100; al recrearse ns1, reinstalar el Lab 7. | `docker compose ps` mostró 5 nodos; `verificar_lab07.sh` → 16 de 16. | Un contenedor puede estar en dos redes, como un servidor con dos tarjetas. |
| 2 | Cómo decide BIND 9.18 qué vista responde y por qué importa el orden (prompt 1). | Recorre las vistas en orden y usa la primera cuyo `match-clients` coincide con la IP de origen; `any` debe ir al final. | Desde el cliente `www.ue-horizonte.example` → 10.10.10.20 y desde externo → 203.0.113.20. | El orden de las vistas es parte de la seguridad. |
| 3 | Qué significa `when using 'view' statements, all zones must be in views`. | Que `named.conf.default-zones` declara zonas fuera de las vistas; hay que quitar ese `include`. | `cat named.conf` mostró que la línea seguía (VS Code no había guardado); la borré con `sed`, `grep -c` → 0 y la instalación siguió. | Leer el archivo real antes de suponer que el cambio está hecho. |
| 4 | Por qué mis cambios de VS Code no llegaban al contenedor. | Las pestañas tenían el punto ● de "sin guardar"; luego apareció un conflicto de guardado porque el archivo se había cambiado en disco con `sed`. | Revisé las pestañas, acepté la versión correcta del conflicto y confirmé con `grep nsec3param` y `grep -c default-zones`. | Guardar y comprobar desde el contenedor, no solo en el editor. |
| 5 | Qué pide BIND sin `inline-signing` (lo probé sobre una copia en `/tmp`). | En 9.18 `dnssec-policy` sobre una zona estática exige `inline-signing yes;` para firmar una copia aparte. | `named-checkconf /tmp/prueba.conf` dio exactamente ese error, sin tocar la configuración real. | Probar errores en copias para no romper el servicio. |
| 6 | La primera instalación terminó con `ls: cannot access … Kue-horizonte.example.+013+*.key`. | El script espera hasta 20 s a que BIND genere la clave y firme; si no llega, falla en el paso del DS. Volver a ejecutarlo. | La segunda ejecución mostró el DS `18104 13 2 0AD9…` y "Listo"; `ls /var/lib/bind/claves` mostró la clave 18104. | Un fallo por tiempo se distingue de un error de configuración. |
| 7 | Explicar cada campo de mi RRSIG y cuándo vence (prompt 2). | Tipo, algoritmo 13, etiquetas, TTL, vencimiento, inicio, etiqueta de clave 18104 y firmante. | El 18104 coincide con el `.key`, el DS y `rndc dnssec -status`; el vencimiento (05/11) coincide con los 14 días de la política. | Relacionar el RRSIG con la clave y la política. |

| # | Qué pregunté | Qué respondió la IA | Cómo lo verifiqué | Qué aprendí |
|---|---|---|---|---|
| 8 | Cómo se "firma" un nombre que no existe. | Con NSEC3: registros firmados que cubren el hueco entre dos hashes de nombres existentes. | `delv … noexiste` → `negative response, fully validated` con registros `NSEC3 1 0 0 -` (0 iteraciones, sin sal). | Por qué RFC 9276 recomienda 0 iteraciones y sin sal. |
| 9 | Cómo cambiar una letra del ancla sin tocar el original. | Un `sed` que escribe una copia en `/tmp/ancla-falsa.conf` y `diff` para mostrar el cambio. | `diff` mostró `jN/9…` → `AN/9…` y `delv` dio `broken trust chain`. | Una sola letra rompe la cadena de confianza. |
| 10 | Cómo calcular en bash los días hasta el vencimiento desde `AAAAMMDDhhmmss` (prompt 3). | Cortar la cadena con `${vence:0:8}` y `${vence:8:2}` …, pasarla a segundos con `date -u -d … +%s`, restar y dividir entre 86 400. | `vigilar_dns.sh` dijo "firmas válidas por 13 días más" (hoy 22/10 → 05/11). | Usar UTC en ambos lados para no desfasar el cálculo. |
| 11 | Por qué el monitoreo avisaba `ns1=2026100701 ns2=2026100703`. | El serial del primario era menor que el del secundario; ns2 nunca copia una zona con serial más bajo. Subirlo a la fecha de hoy. | `sed` a 2026102201 + `rndc reload horizonte.internal in interna`; los dos `dig SOA` → 2026102201 y el verificador → 16 de 16. | El serial solo puede subir; con vistas, `rndc reload` lleva `in <vista>`. |
| 12 | Qué protege DNSSEC que HTTPS no, y por qué se usa poco. | DNSSEC autentica las respuestas DNS (y la inexistencia) para todos los registros; HTTPS protege la conexión. Se usa poco por la complejidad y el riesgo operativo. | Lo contrasté con RFC 4033 y con la documentación de ISC sobre `dnssec-policy`. | Las dos capas se complementan; no se reemplazan. |

## Conclusión

La IA me sirvió para entender las salidas (vistas, RRSIG, NSEC3, delv) y para diagnosticar tres tropiezos: el `include` de `default-zones` que VS Code no había guardado, la clave que tardó en generarse y el serial menor que el del secundario. En cada caso la causa la confirmé con un comando antes de cambiar algo. La clave privada nunca salió de ns1, la validación nunca se apagó y la firma la hace `dnssec-policy` sola. Resultado final: **16 de 16**, y la misma consulta da una respuesta distinta al colegio y a internet.